Implementation Report

Adaptive Workplace Dynamics Simulator

Student name: *[to be completed]*

June 17, 2026

Abstract

This document describes the Adaptive Workplace Dynamics Simulator (AWDS) prototype, a Python application with a Streamlit interface for synthetic simulation of workplace dynamics. The report presents the implemented features, the model, the technical structure, encountered difficulties, limitations, and future development directions. Numerical results are intended to be completed after the final scenarios are run.

Contents

1 Prototype Description

AWDS is a software prototype that simulates an organization made of employee agents. Each agent has an internal dynamic state, including stress, burnout, productivity, emotional state, energy, job satisfaction, work-life balance, and engagement.

The application is built as an exploratory tool. Its purpose is not to prove real organizational relationships, but to observe synthetic dynamics generated by a configurable model. Therefore, the results should be interpreted as model outputs, not empirical data.

The interface allows the user to configure parameters, run simulations, view graphs, compare scenarios, and export results for later use in a report.

2 Implemented Features

The prototype includes the following main features:

- configuration for number of agents, number of simulated days, and random seed;
- factory presets for predefined organizational scenarios;
- local user presets saved in `user_presets.json`;
- explicit preset application button to avoid accidental parameter changes;
- saving the current configuration as a new preset;
- sticky controls for run, pause/resume, and clearing the displayed run;
- visible slider range editing through the *Advanced slider ranges* panel;
- live graphs for stress, burnout, productivity, and secondary indicators;
- engine selector for the custom engine, the Mesa engine, or both engines in sequence;
- scenario comparison using the same seed and a selected comparison engine;
- direct comparison between engine outputs when both engines are selected;
- completed run history on the export page;
- search in export history by scenario, timestamp, seed, day, number of agents, or ID;
- export to JSON, CSV, TXT, PNG, and ZIP;
- startup scripts for Unix/macOS and Windows: `run.sh` and `run.bat`.

3 Simulation Model

Each simulation step represents one simulated workday. On each day, the model applies organizational factors, personal factors, social influence, random events, and update rules for each agent.

3.1 Dynamic Variables

Agents have the following main dynamic variables:

- **stress**: stress level, normalized between 0 and 1;
- **burnout**: burnout accumulation, normalized between 0 and 1;
- **productivity**: current productivity, normalized between 0 and 1;
- **emotion**: emotional state, between -1 and 1;
- **energy**: energy or recovery capacity, between 0 and 1;
- **satisfaction**: job satisfaction, between 0 and 1;
- **work_life_balance**: work-life balance, between 0 and 1;
- **engagement**: employee engagement, between 0 and 1.

3.2 Static Factors

Each agent is initialized with different personal traits: role, resilience, ambition, need for social support, commute sensitivity, family load, health variability, baseline productivity, stress sensitivity, and recovery rate.

3.3 General Logic

The model uses simple and transparent equations. Stress increases based on work demands, external pressure, and negative events, but decreases based on resources, recovery, and resilience. Burnout accumulates more slowly than stress and is influenced by sustained high stress. Productivity is not modeled as a linear "stress is always bad" relationship. The model includes an optimal pressure effect: moderate stress may support productivity in the short term, while high stress and burnout reduce productivity.

3.4 Turnover

In the application, *turnover* means the cumulative number of simulated employees who leave the organization. An agent may leave when burnout or stress exceeds configured thresholds. If replacement is enabled, a new agent joins after a configured delay and temporarily receives an onboarding productivity penalty.

3.5 Reproducibility

The custom engine uses `numpy.random.default_rng(seed)`, while the Mesa engine receives the same configured seed through the Mesa model. The same seed, configuration, and selected engine produce the same result series for that engine. The same seed is not expected to produce identical trajectories across the custom and Mesa engines, because the implementation and activation order are different. This property is useful for comparing scenarios under controlled conditions while also comparing how stable the observed tendencies are across engine implementations.

4 Technical Implementation

The application is implemented in Python. The web interface is built with Streamlit, and numerical calculations are handled with NumPy. Data is organized and exported with Pandas, while exportable charts are generated with Matplotlib. Plotly is used for interactive dashboard charts when available. The Mesa implementation uses Mesa model and agent classes together with NetworkX-backed network space.

4.1 Project Structure

```
app.py
run.sh
run.bat
simulation/
  config.py
  agent.py
  engine.py
  mesa_engine.py
  engines.py
  scenarios.py
  metrics.py
  exports.py
  utils.py
exports/
requirements.txt
```

4.2 Simulation Engine Architecture

The prototype now separates the simulation logic behind an engine interface. An engine exposes the same basic operations: advancing one day, running until completion, returning a live snapshot, and returning the final result. The dashboard uses `engines.py` to map the selected engine name to the implementation that should run. This makes the interface independent from the exact engine implementation.

The custom engine, implemented in `engine.py`, is a lightweight local implementation. It directly manages the list of employee agents, the social network, random events, daily state updates, turnover, replacement, and aggregate metrics. Its main advantage is transparency: the update equations and output format are easy to inspect and tune.

4.3 Mesa Alternative

Mesa has also been added as an alternative engine in `mesa_engine.py`. In this version, the workplace is represented as a Mesa model, employees are Mesa agents, and relationships are stored in a network grid backed by NetworkX. The Mesa engine preserves the same result fields as the custom engine, so the dashboard, exports, and comparison tables can treat both engines uniformly.

The goal of adding Mesa is not to make the outputs identical. Instead, the goal is to compare scenario tendencies across two implementations. If the custom and Mesa engines show similar directions under the same preset, seed, and run controls, the model

behavior is easier to interpret. If the outputs differ, that difference helps identify which assumptions or implementation details need closer research.

5 Scenarios and Results

This section is a skeleton to be completed after running the final scenarios and exporting the graphs.

The comparison workflow is central to the prototype. Scenarios can be compared with the custom engine, with the Mesa engine, or with both engines. This gives a clearer view of whether the research focus should be on a scenario effect itself or on differences introduced by the simulation engine.

5.1 Experimental Configuration

- Number of agents: *[to be completed]*
- Number of simulated days: *[to be completed]*
- Seed used: *[to be completed]*
- Engine or engines compared: *[to be completed]*
- Compared scenarios: *[to be completed]*
- Parameters changed from presets: *[to be completed]*

5.2 Comparison Table

Scenario	Engine	Final stress	Final burnout	Final productivity	Turnover	Cumulative burnout
[Scenario 1]	*[Engine]*	–	–	–	–	–
[Scenario 2]	*[Engine]*	–	–	–	–	–
[Scenario 3]	*[Engine]*	–	–	–	–	–

Table 1: Synthetic comparative results obtained from AWDS simulations.

5.3 Graphs

Insert exported application graphs here.

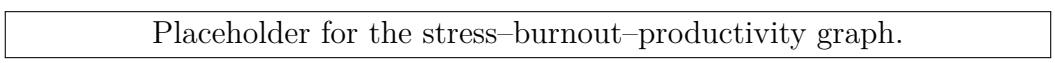

Figure 1: Evolution of stress, burnout, and productivity for scenario *[to be completed]*.

5.4 Interpretation

[To be completed after running the final scenarios. The interpretation should describe synthetic model dynamics, compare custom and Mesa tendencies when both engines are used, and avoid presenting the outputs as empirical findings.]

6 Encountered Difficulties

The main difficulty was designing a model that was expressive enough not to reduce workplace dynamics to trivial rules such as “stress is bad” and “free time is good”. For the simulation to be useful, the model had to allow more nuanced situations: for example, moderate pressure may increase productivity in the short term, while high and sustained pressure may lead to burnout and lower productivity.

Another difficulty was balancing the parameters so scenarios produce visible differences without all runs immediately reaching extremes. It was necessary to separate stress, burnout, energy, satisfaction, and productivity, because these variables do not all move at the same speed and do not respond identically to the same factors.

Interface configurability was also a practical challenge, because the number of parameters is large. Streamlit significantly simplified the UI side by enabling fast construction of configuration panels, charts, and controls. Still, the controls needed clear grouping, explicit presets, parameter explanations, and an export system that is easy to use.

Exporting results was treated as part of the user experience: the user should be able to return to runs, search them, identify each run by ID and timestamp, and download data in formats suitable for the report.

Adding a second engine introduced an additional implementation challenge: both engines had to produce the same output schema so that charts, exports, run history, and comparison tables could reuse the same code. This was important because the research value comes from comparing results, not from maintaining separate workflows for each implementation.

7 Limitations

The most important limitation is that the model uses synthetic data. Parameters and equations are configurable and reproducible, but they are not calibrated on real empirical data. For this reason, results can be used to discuss the internal behavior of the model, not to draw conclusions about real companies.

The model simplifies many aspects of organizational environments: social relationships are represented through a static network, events are modeled probabilistically, and agent traits are generated randomly from simple distributions. These decisions are appropriate for a prototype, but they limit simulation realism.

8 Future Directions

The most important next step would be obtaining empirical data for model calibration. One possible direction is creating a questionnaire distributed to people working in different companies, collecting information about perceived stress, workload, managerial support, autonomy, work-life balance, satisfaction, and intention to leave the organization.

The goal of such a questionnaire would be to turn subjective experiences into approximate values, in other words to “put a number on a feeling”. These values could be used to adjust model parameters, compare synthetic scenarios with real responses, and identify variables that should be represented more accurately.

Other development directions include:

- calibrating parameters based on collected responses;

- extending the model with teams, departments, and more detailed organizational roles;
- sensitivity analysis to identify which parameters influence results the most;
- systematic comparison between custom-engine and Mesa-engine outputs;
- persistent storage of runs in a database;
- improved visualizations and automatically generated reports;
- partial validation by comparing synthetic tendencies with anonymized real data.

9 Video Demo

The video demo will present the main application flow:

1. starting the application;
2. applying a factory preset;
3. manually adjusting parameters;
4. saving a local preset;
5. selecting the custom engine, Mesa engine, or both engines;
6. running a simulation;
7. pausing/resuming a live run;
8. comparing multiple scenarios with the selected comparison engine;
9. exporting results from an export drawer;
10. downloading JSON, CSV, TXT, PNG, and ZIP files.

Video demo link: *[to be completed]*